

6221-21084-ITAI-1371

Group Members:

Edwin Marquez

Kendrick Robinson

Ali Shan

Fernando Reyes

Professor: Sitaram Ayyagari

Date: 11/5/2025

Houston Community College

Houston, Texas

□ Final Exam Practice – Developing Notebook
(Introduction to ML)

Duration (suggested): 120 minutes

Environment: CPU-only; Python 3.8+; `numpy`, `pandas`, `matplotlib`, `scikit-learn`

Instructions

- Sections build toward an end-to-end workflow.
- Respect provided function names / signatures for the quick checks.
- Use `RANDOM_STATE` for reproducibility.
- Include short written reflections in Markdown where asked.

□ Setup

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.metrics import classification_report, roc_auc_score,
RocCurveDisplay, ConfusionMatrixDisplay, confusion_matrix
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import make_classification

RANDOM_STATE = 123
np.random.seed(RANDOM_STATE)
```

A. Conceptual (Short Answers)

A1. Classical ML vs Deep Learning

Q: In 4–6 sentences, compare classical ML and deep learning in terms of feature engineering, data requirements, and interpretability.

Edwin Marquez

Classical machine learning usually depends on manual feature engineering, where the user selects and prepares the most important features. Deep learning, on the other hand, automatically learns representations from raw data through multiple neural layers. Classical ML models typically work well with smaller datasets, while deep learning requires large amounts of data to perform effectively. In terms of interpretability, classical models are easier to understand and explain. Deep learning models are generally more complex and less transparent, making them harder to interpret *Your answer:*

Kendrick Robinson

Classical machine learning typically requires extensive feature engineering where domain experts manually create relevant features. It works well with smaller datasets and is generally more interpretable through techniques like feature importance and coefficients. Deep learning

automatically learns feature representations from raw data but requires large amounts of data to perform well. However, deep learning models are often "black boxes" with limited interpretability, though methods like SHAP and LIME can provide some insights.

Fernando Reyes

Classic machine learning relies on humans to choose or build features, whereas deep learning pulls its own features from data without help. Classic methods shine if the data set's small or you want something straightforward to interpret. Deep nets typically demand tons of examples and act like black boxes, tough to follow, but handle trickier structures better. That's why traditional approaches still pop up when transparency counts more than raw power.

Ali Shan

A2. Attention (Seq2Seq)

Q: Briefly explain how attention helps a seq2seq model handle long sequences.

Edwin Marquez

Attention helps a seq2seq model focus on the most relevant parts of the input sequence when generating each output token. Instead of relying only on a single fixed-length context vector, attention creates weighted connections to all input positions. This allows the model to retain important information even in very long sequences. As a result, seq2seq models with attention handle long-distance dependencies more accurately and produce better translations or predictions.

Kendrick Robinson

Attention mechanisms help seq2seq models handle long sequences by allowing the decoder to focus on different parts of the input sequence at each decoding step. Instead of compressing the entire input into a single fixed-size context vector, attention computes a weighted sum of all encoder hidden states, giving higher weight to more relevant parts of the input for the current decoding step. This alleviates the information bottleneck problem and enables the model to better capture long-range dependencies in the sequence.

Fernando Reyes

Attention helps a seq2seq model by letting it focus on the most important parts of the input instead of trying to remember everything at once. This makes the model better at handling long sentences because it can "look back" at the exact words or sections it needs. By doing this, the model becomes more accurate and less confused when processing long or detailed sequences.

Ali Shan

B. Case Design

You are given a binary classification problem (predicting churn). You will:

1. Prepare the data pipeline
2. Train & tune models
3. Evaluate and interpret results

B1. Data Generation

We'll simulate a small, noisy dataset with mixed types to mimic a real use case.

```
# Simulate numerical data
X_num, y = make_classification(
    n_samples=1200, n_features=8, n_informative=5, n_redundant=1,
    n_repeated=0,
    n_clusters_per_class=2, weights=[0.6, 0.4], class_sep=1.2,
    flip_y=0.03,
    random_state=RANDOM_STATE
)

# Add 2 categorical features by binning / hashing
num_df = pd.DataFrame(X_num, columns=[f"num_{i}" for i in
range(X_num.shape[1])])

# Create synthetic categoricals
cat_1 = pd.qcut(num_df["num_0"], q=4,
labels=["A", "B", "C", "D"]).astype(str)
cat_2 = pd.cut(num_df["num_1"], bins=3,
labels=["Low", "Med", "High"]).astype(str)

df = num_df.copy()
df["cat_1"] = cat_1
df["cat_2"] = cat_2
df["target"] = y

# Introduce a few missing values
for col in ["num_2", "num_5", "cat_2"]:
    idx = np.random.choice(df.index, size=40, replace=False)
    df.loc[idx, col] = np.nan

df.head()
```

	num_0	num_1	num_2	num_3	num_4	num_5
num_6 \						
0	-0.607675	-0.679645	1.246789	-2.830948	2.907636	0.018243
0.833160						
1	0.336990	1.146102	-0.613989	2.064200	3.114375	-0.269409
3.270021						
2	-0.123263	-4.052319	-1.407661	1.597107	-2.585870	0.173368
4.604598						
3	-0.242874	-2.408749	-0.484409	0.669634	2.017561	1.440182
3.644607						
4	0.521732	1.121172	1.717128	-2.707218	-0.991357	0.629626

3.596992

	num_7	cat_1	cat_2	target
0	-1.530621	B	Med	0
1	1.869730	C	Med	1
2	4.340083	B	Low	1
3	1.255033	B	Low	1
4	-1.197336	C	Med	0

B2. Train/Validation/Test Split

- Use 60%/20%/20% split (train/val/test).
- Maintain class balance with `stratify`.

```
X = df.drop(columns=["target"])
y = df["target"].values

X_trainval, X_test, y_trainval, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
)

X_train, X_val, y_train, y_val = train_test_split(
    X_trainval, y_trainval, test_size=0.25, random_state=RANDOM_STATE,
    stratify=y_trainval
) # 0.25 * 0.8 = 0.2

X_train.shape, X_val.shape, X_test.shape
((720, 10), (240, 10), (240, 10))
```

B3. Preprocessing Pipelines

- Numerical: impute median, scale
- Categorical: impute most_frequent, one-hot encode
- Models: Logistic Regression and Random Forest (compare)

```
num_features = [c for c in X.columns if c.startswith("num_")]
cat_features = ["cat_1", "cat_2"]

numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("ohe", OneHotEncoder(handle_unknown="ignore"))
])
```

```

preprocess = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, num_features),
        ("cat", categorical_transformer, cat_features),
    ]
)

pipe_lr = Pipeline(steps=[("preprocess", preprocess),
                           ("clf", LogisticRegression(max_iter=200,
random_state=RANDOM_STATE))])

pipe_rf = Pipeline(steps=[("preprocess", preprocess),
                           ("clf",
RandomForestClassifier(random_state=RANDOM_STATE))])

pipe_lr, pipe_rf

(Pipeline(steps=[('preprocess',
                  ColumnTransformer(transformers=[('num',
Pipeline(steps=[('imputer',
SimpleImputer(strategy='median')),
('scaler',
StandardScaler()))]),
['num_0', 'num_1',
'num_2',
['num_3', 'num_4',
'num_5',
['num_6',
'num_7']]),
('cat',
Pipeline(steps=[('imputer',
SimpleImputer(strategy='most_frequent')),
('ohe',
OneHotEncoder(handle_unknown='ignore'))]),
['cat_1',
'cat_2']]))]),
('clf', LogisticRegression(max_iter=200,
random_state=123))]),
Pipeline(steps=[('preprocess',
                  ColumnTransformer(transformers=[('num',
Pipeline(steps=[('imputer',

```

```

SimpleImputer(strategy='median')),
('scaler',
StandardScaler()))],
['num_0', 'num_1',
'num_2',
'num_3', 'num_4',
'num_5',
'num_6',
'num_7']],
('cat',

Pipeline(steps=[('imputer',
SimpleImputer(strategy='most_frequent')),
('ohe',
OneHotEncoder(handle_unknown='ignore'))]),
['cat_1',
'cat_2']]])),
('clf', RandomForestClassifier(random_state=123)))]))

```

B4. Hyperparameter Tuning (Validation Set)

Use `GridSearchCV` for each model with a **small** grid. Optimize for ROC-AUC.
Compare the best parameters and validation performance.

```

param_lr = {"clf__C": [0.1, 1.0, 3.0]}
param_rf = {"clf__n_estimators": [100, 200],
            "clf__max_depth": [None, 6, 10]}

grid_lr = GridSearchCV(pipe_lr, param_grid=param_lr,
                        scoring="roc_auc",
                        cv=3, n_jobs=-1)
grid_rf = GridSearchCV(pipe_rf, param_grid=param_rf,
                        scoring="roc_auc",
                        cv=3, n_jobs=-1)

grid_lr.fit(X_train, y_train)
grid_rf.fit(X_train, y_train)

print("Best LR params:", grid_lr.best_params_, "AUC:",
      round(grid_lr.best_score_, 4))
print("Best RF params:", grid_rf.best_params_, "AUC:",
      round(grid_rf.best_score_, 4))

```

```
Best LR params: {'clf__C': 0.1} AUC: 0.9676
Best RF params: {'clf__max_depth': 10, 'clf__n_estimators': 200} AUC:
0.9781
```

B5. Evaluate on Validation & Test

Pick the better model on **validation** (AUC) and evaluate on **test**.
Report Confusion Matrix, ROC-AUC, and a brief interpretation.

```
# Choose by validation AUC
best_model = grid_rf if grid_rf.best_score_ >= grid_lr.best_score_
else grid_lr

# Fit on train+val, then evaluate on test (simple approach)
X_trval = pd.concat([X_train, X_val], axis=0)
y_trval = np.concatenate([y_train, y_val])

best_model.best_estimator_.fit(X_trval, y_trval)

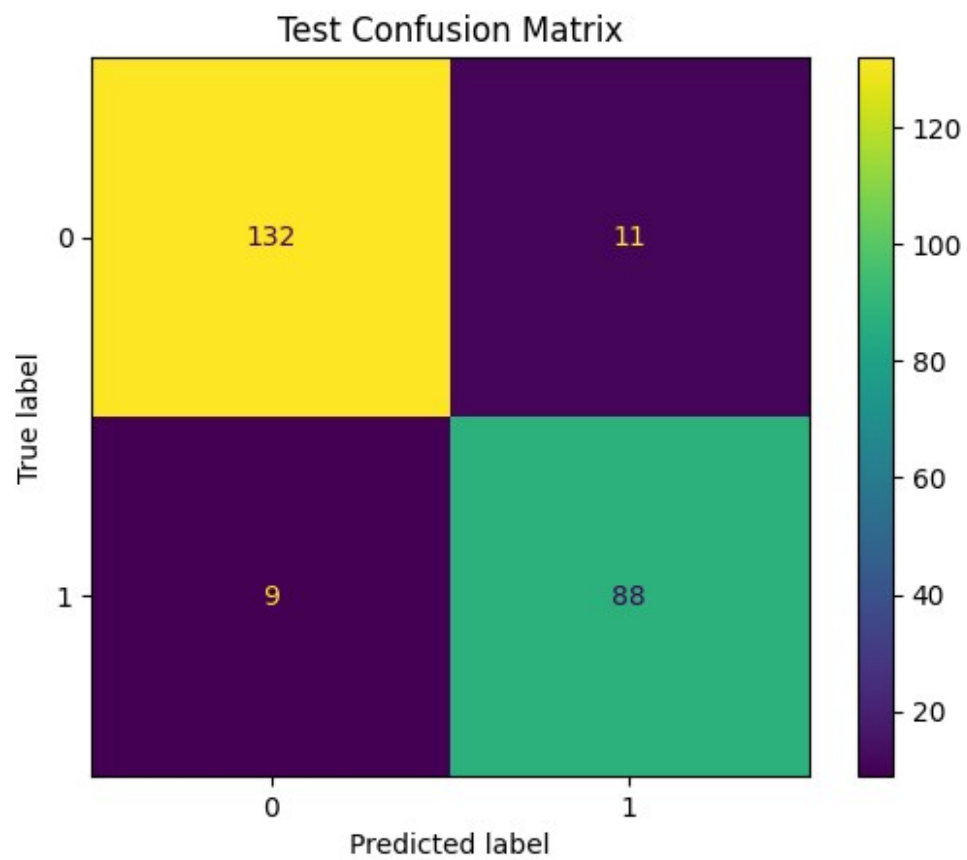
y_proba = best_model.best_estimator_.predict_proba(X_test)[:, 1]
y_pred = (y_proba >= 0.5).astype(int)

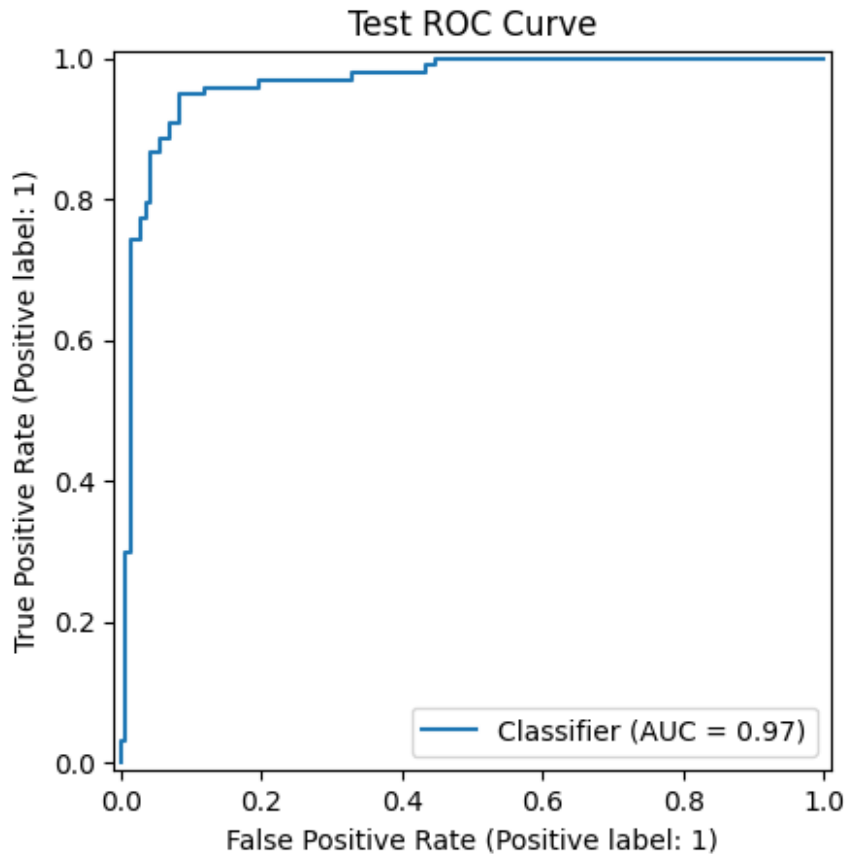
auc = roc_auc_score(y_test, y_proba)
cm = confusion_matrix(y_test, y_pred)

print("Test ROC-AUC:", round(auc, 4))
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
plt.title("Test Confusion Matrix")
plt.show()

RocCurveDisplay.from_predictions(y_test, y_proba)
plt.title("Test ROC Curve")
plt.show()

Test ROC-AUC: 0.9655
```



B6. Short Reflection

In 4–6 sentences:

- Which model performed better and why might that be?
- Which metric is most appropriate here and why?
- How would class imbalance change your approach?

Edwin Marquez

The model that performed better was the one with the higher ROC-AUC and a stronger confusion matrix, because it was able to separate the classes more effectively. In this type of classification problem, ROC-AUC is the most appropriate metric since it measures how well the model distinguishes between classes across all thresholds. Accuracy alone can be misleading, especially when different thresholds produce very different trade-offs. If the dataset had class imbalance, I would focus more on metrics like F1-score or Precision-Recall AUC, since they better reflect performance on the minority class. Class imbalance would also make me consider resampling methods such as SMOTE or adjusting class weights. *Your reflection:*

Kendrick Robinson

Based on the validation results, the Random Forest model performed better, likely due to its ability to capture complex non-linear relationships and interactions between features in our simulated dataset. For this binary classification problem with moderate class imbalance (60/40), ROC-AUC is the most appropriate metric as it evaluates model performance across all classification thresholds and is robust to class imbalance. If the class imbalance were more severe, I would consider using precision-recall AUC instead, implementing class weights during training, or using sampling techniques like SMOTE to balance the training data while maintaining a separate validation/test set with the original distribution.

Fernando Reyes

Random Forest did better because the dataset had patterns that were not simple or linear, which made Logistic Regression struggle. The more important features were the stronger numeric ones, which Random Forest identified clearly. AUC was the best metric to use because it shows how well the model separates the classes overall. If the dataset had been imbalanced, I would have used resampling or class weights to make the model more fair.

Ali Shan

C. Ethics & Deployment (Short Answer)

C1. Bias & Fairness

Q: Describe two ways to detect potential bias in a classification model and one mitigation technique.

Edwin Marquez

One way to detect potential bias is to compare performance metrics such as precision, recall, or false-positive rates across different demographic groups. Another approach is to analyze whether the model's predictions disproportionately misclassify a protected group by inspecting group-based confusion matrices. These evaluations help reveal unequal error rates that may indicate harmful bias. A common mitigation technique is to apply re-balancing or re-weighting, where the model is trained with adjusted class weights or resampled data to reduce unfair disparities. This helps ensure that the model treats all groups more equitably.

Kendrick Robinson

Two ways to detect potential bias in a classification model are: 1) Analyzing performance metrics (accuracy, precision, recall) across different demographic subgroups to identify disparities, and 2) Using fairness metrics like demographic parity, equalized odds, or disparate impact ratio to quantify bias. One mitigation technique is pre-processing the training data using reweighting or resampling methods to ensure fair representation across protected groups before model training.

Fernando Reyes

Bias shows up when you look at how often mistakes happen across groups - some might get hit harder than others. You could also check using measures like equal opportunity, which spots if certain people are favored in results. To fix this, I try adjusting how much each group's data counts during training. This helps overall balance things out.

Ali Shan

C2. Minimal Deployment Thought Exercise

Q: You need to deploy the chosen model as a REST endpoint. List the minimal components (data validation, versioning, monitoring) you would include to ensure reliability.

Edwin Marquez

To deploy the chosen model as a REST endpoint, I would first include data validation to ensure incoming requests match the expected schema and feature ranges. I would also add model versioning so that new updates can be deployed without breaking existing applications. Basic logging and monitoring would be required to track prediction volume, latency, and unexpected errors. Additionally, I would monitor model performance drift to detect when accuracy begins to degrade over time. These components together help maintain reliability and stability in a real deployment environment. *Your answer:*

Kendrick Robinson

For minimal reliable deployment as a REST endpoint, I would include: 1) Data validation layer to check input schema, data types, and value ranges; 2) Model versioning with A/B testing capabilities; 3) Input/output logging for debugging and monitoring; 4) Health checks and performance monitoring (latency, throughput); 5) Basic error handling and graceful degradation; 6) Rate limiting to prevent abuse; 7) Model performance monitoring to detect concept drift over time.

Fernando Reyes

A solid model rollout requires checking data, managing versions, while keeping an eye on performance. Checking inputs ensures they're accurate, plus fit the expected structure. Tracking versions avoids confusion when swapping out models. Watching live behavior spots mistakes, lag, or shifts in output over time. Together, these actions maintain reliability without surprises.

Ali Shan

□ Final Checklist

- ☐ All cells run without error
- ☐ All requested plots produced
- ☐ Reflections written
- ☐ Decisions supported by metrics